The Hidden Cost of Storage Mismatches: Why Faster I/O Per Task Can Slow Down Entire Workflows

Gnosis Research Center @ ILLINOIS TECH

Contact: Meng Tang (mtang11@hawk.iit.edu)
Advisors: Anthony Kougkas, Xian-He Sun
Collaborators: Luanzheng Guo (PNNL), Nathan R. Tallent (PNNL)

Pacific Northwest NATIONAL LABORATORY

Introduction & Motivation

- HPC workflows exchange intermediate data through tiered storage: parallel FS, node-local SSDs, burst buffers
- I/O often dominates runtime in data-intensive workflows
- Existing schedulers treat storage selection per-task or workflow DAG per-partition
- Per-task throughput gains do not always translate to workflow-level speedup

Producer-Consumer Storage Mismatch

- Reads and writes have different fastest storage
- Per-task optimum $\neq$ pair optimum
- Per-task pick: **up to 6.8**$\times$ slowdown vs. storage-aware
- Mismatch penalty $>$ task compute cost

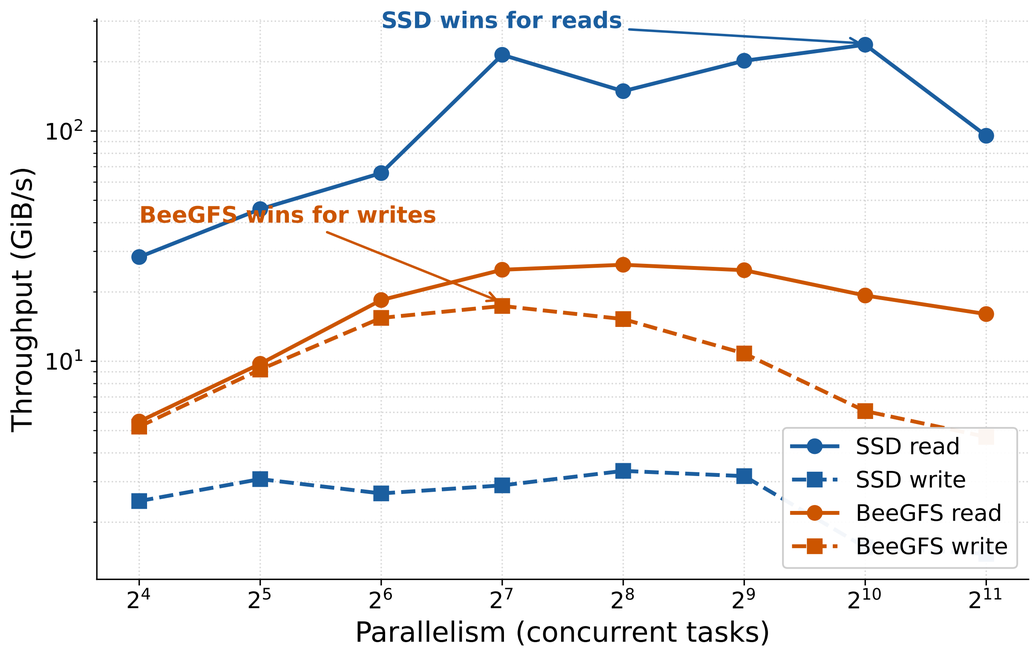

Figure 1: Throughput vs. parallelism (1 MB transfers, 4 nodes). Reads scale faster on **SSD**; writes scale faster on **BeeGFS**. Producers (writers) and consumers (readers) therefore prefer opposite tiers.

Cross-Tier Data Movement Scales Differently

- Read/write: scale predictably to saturation
- **Cross-tier copy:** plateaus or degrades earlier

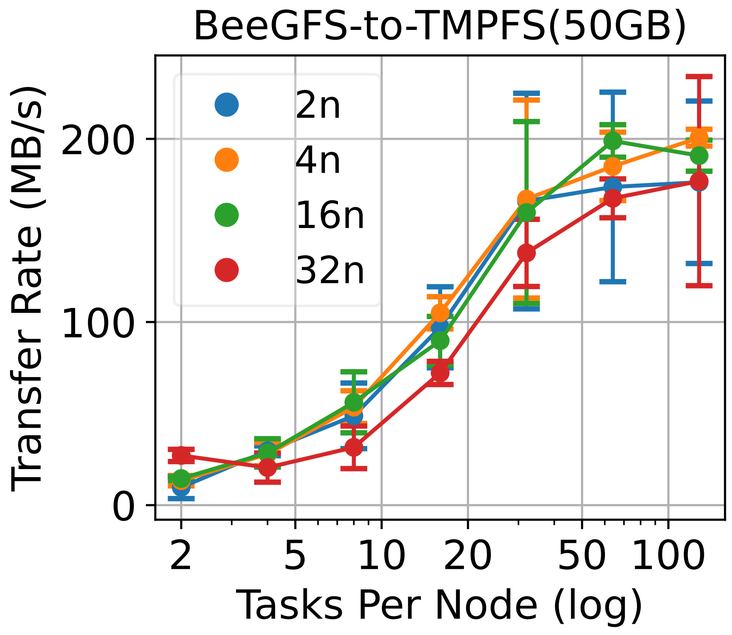

Figure 2: BeeGFS→tmpfs copy plateaus where reads and writes still scale (50 GB).

Key Takeaways

- Per-task throughput $\neq$ workflow throughput
- Cross-tier movement scales differently than read/write
- Producer-consumer dependencies = first-class scheduling constraint

Case Study

Storage tiers: BeeGFS (shared PFS), NVMe SSD (node-local), tmpfs (in-memory)

One-time profiling cost: 30–36 node-hours of IOR across 3 storage tiers (transfer 64 B–1 GB, parallelism 2–128K); profiles reusable across workflows

1000 Genomes (genomics): 5 stages, **300-way parallel**, $\sim$**50 GB** read I/O, 10 chromosomes

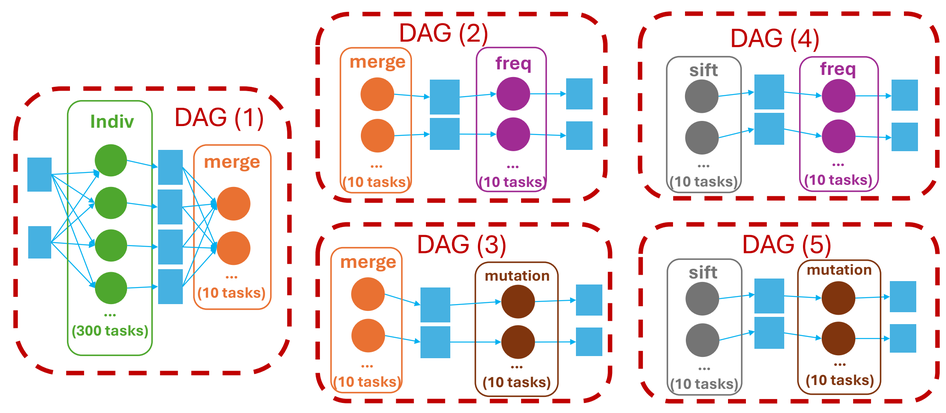

Figure 3: 1000 Genomes producer-consumer DAG.

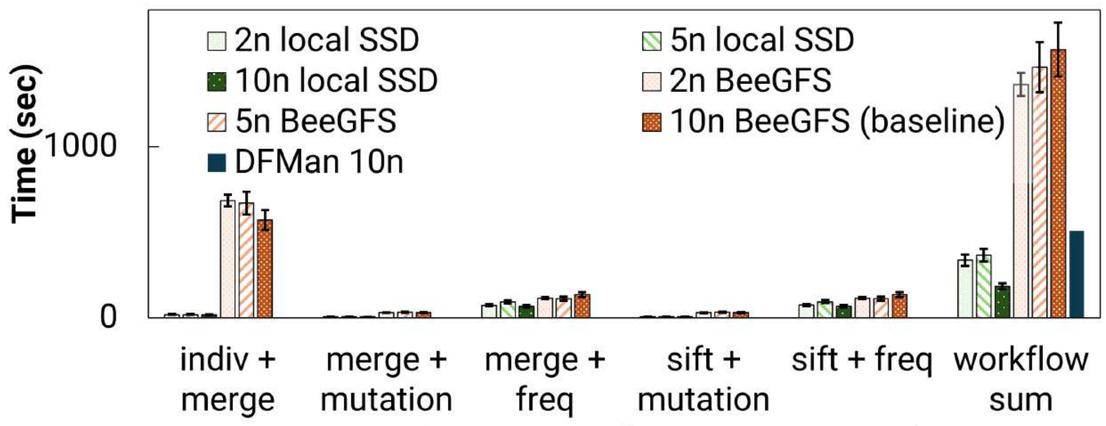

Figure 4: 1000 Genomes runtime per producer-consumer pair across storage and node count. **indiv+merge** is $>$**30**$\times$ slower on BeeGFS than on local SSD.

PyflexTRKR (climate): 9 stages, 96-way parallel, $>$**10M I/O ops**, $\sim$25 GB data

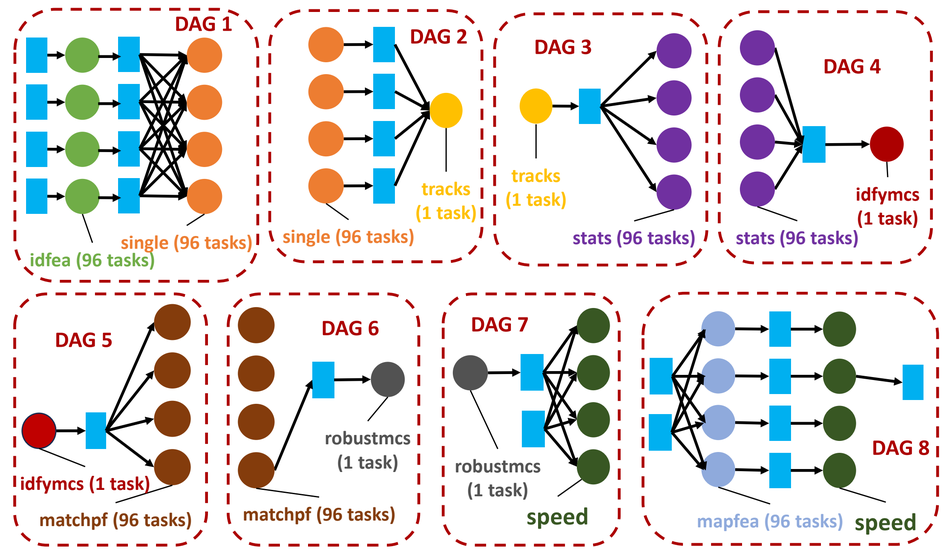

Figure 5: PyflexTRKR producer-consumer DAG.

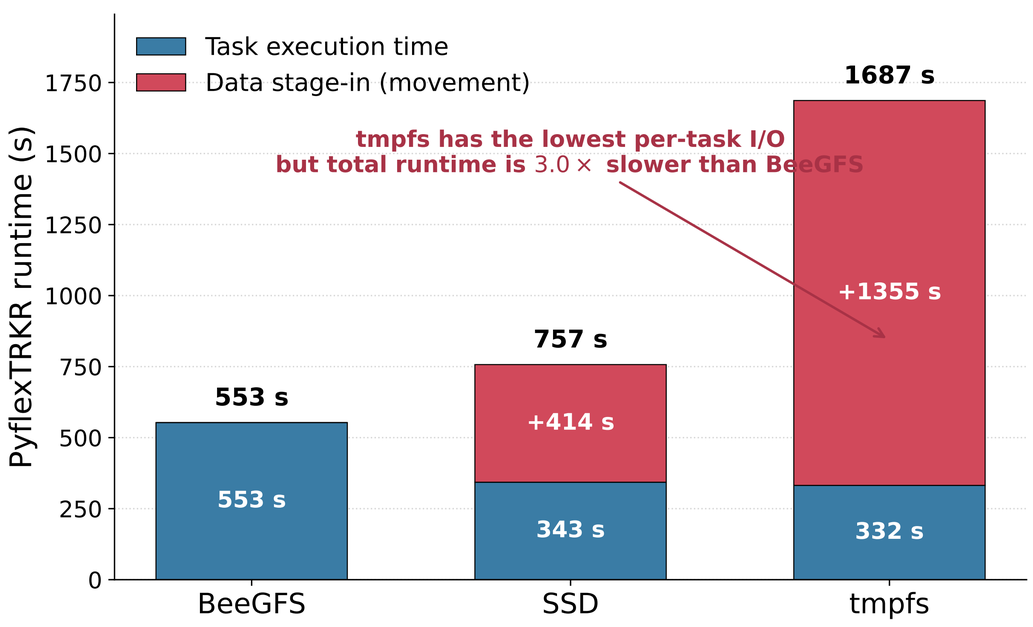

Figure 6: PyflexTRKR runtime at 8 nodes split into **task execution time** (blue) and **data stage-in time** (red). tmpfs has the **lowest per-task** time, yet total runtime is **3**$\times$ slower than BeeGFS because staging dominates.